AI VIET NAM
@aivietnam.edu.vn

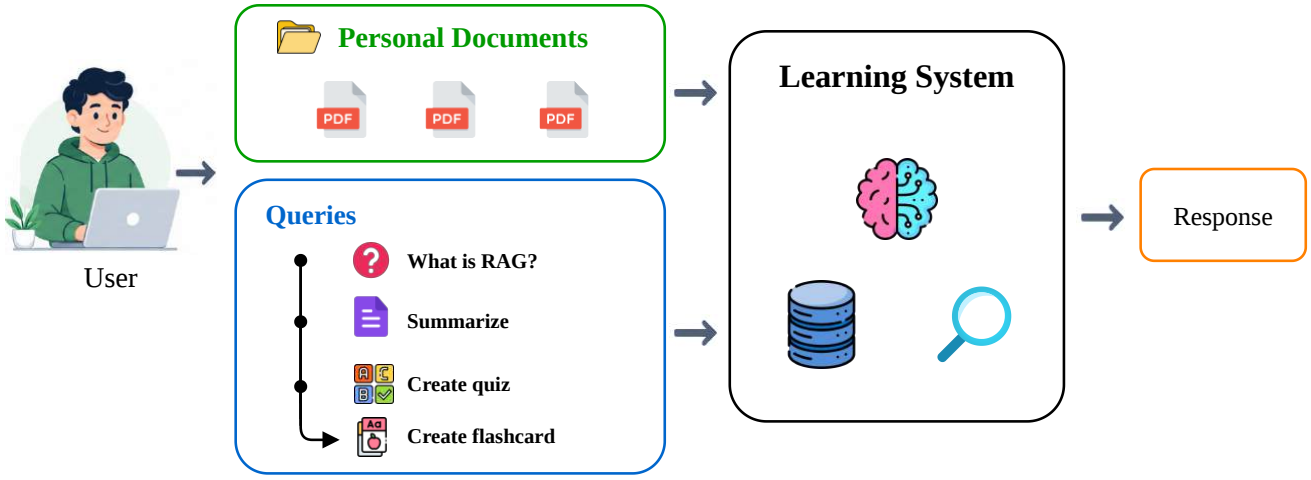
AI VIET NAM – AI COURSE 2025

Project: Building a Simple NotebookLM

Dương Trường Bình Tô Phát Đạt Dương Đình Thắng Đinh Quang Vinh

I. Dẫn nhập

Các mô hình ngôn ngữ lớn (LLM) đã và đang được ứng dụng rộng rãi trong học tập, nghiên cứu và khai thác tri thức từ tài liệu. Tuy nhiên, khi áp dụng vào các bài toán giáo dục, tính chính xác, khả năng kiểm chứng và mức độ bám sát tài liệu nguồn là những yêu cầu rất quan trọng. Nếu chỉ dựa vào tri thức có sẵn trong mô hình, hệ thống có thể sinh ra thông tin sai lệch hoặc không có căn cứ, thường được gọi là hiện tượng "ảo giác" hay còn gọi là hallucination.



Hình 1: Minh hoạ tổng quan dự án cùng các chức năng chính.

Project này mô phỏng một hệ thống hỗ trợ học tập dựa trên tài liệu cá nhân. Thay vì để LLM trả lời hoàn toàn từ tri thức nội tại, hệ thống sử dụng kiến trúc retrieval-augmented generation (RAG) để truy xuất các đoạn nội dung liên quan từ tài liệu đã được nạp trước, sau đó đưa các ngữ cảnh này vào prompt để sinh phản hồi. Cách tiếp cận này giúp câu trả lời có cơ sở rõ ràng hơn, giảm rủi ro hallucination và hỗ trợ người học kiểm tra lại nguồn thông tin.

Mục lục

I. Dẫn nhập **1**
I.1. Mục tiêu 3
I.2. Input và Output 3
II. Nội dung project **4**
II.1. Tiền xử lý và lưu trữ dữ liệu 4
II.2. Pipeline truy xuất và tạo sinh RAG 4
II.3. Chức năng học tập và giao diện sử dụng 5
II.4. Đánh giá và tối ưu hệ thống 5
III. Cài đặt chương trình **6**
III.1. Cấu trúc mã nguồn của project 6
III.2. Luồng hoạt động tổng thể 7
III.3. Cấu hình hệ thống và schema dữ liệu 7
III.4. Nạp tài liệu, chia chunk và lưu trữ vector 11
III.5. Chuẩn hóa bộ lọc và truy xuất ngữ cảnh 15
III.6. Cài đặt pipeline RAG 16
III.7. Tích hợp các backend LLM 19
III.8. Các chức năng học tập: tóm tắt, quiz và flashcards 20
III.9. Xuất kết quả học tập 24
III.10. API, CLI và giao diện người dùng 25
IV. Thực nghiệm **28**
IV.1. Retrieval-augmented generation assessment 28
IV.2. Chiến lược thực nghiệm 30
IV.3. Thực nghiệm: Chunking 30
IV.4. Thực nghiệm: Reranking 34
Phụ lục **38**

Như được minh họa trong Hình 1, ngoài chức năng hỏi đáp trên tài liệu, hệ thống còn cung cấp các chức năng học tập như tóm tắt nội dung, tạo câu hỏi trắc nghiệm (Quiz) và tạo thẻ ghi nhớ (Flashcards). Bên cạnh đó, project cũng tích hợp phần đánh giá bằng Ragas để so sánh các chiến lược chunking và kiểm tra tác động của reranking đến chất lượng truy xuất.

I.1. Mục tiêu

Mục tiêu của project là xây dựng một hệ thống học tập như một phiên bản NotebookLM đơn giản, có khả năng tiếp nhận các tài liệu học tập và hỗ trợ người dùng khai thác nội dung tài liệu thông qua LLM. Cụ thể, hệ thống đáp ứng các yêu cầu sau:

1. **Hỏi đáp dựa trên tài liệu:** Hệ thống cần truy xuất các đoạn nội dung liên quan từ tài liệu đã index, sau đó sinh câu trả lời dựa trên ngữ cảnh được cung cấp và kèm theo thông tin trích dẫn nguồn.
2. **Tóm tắt và hệ thống hóa kiến thức:** Hệ thống hỗ trợ tóm tắt theo phạm vi tài liệu, bộ lọc hoặc truy vấn cụ thể. Với tài liệu dài, hệ thống có thể sử dụng chiến lược map-reduce để tổng hợp nội dung theo từng nhóm chunk.
3. **Tạo học liệu tương tác:** Hệ thống tự động tạo câu hỏi trắc nghiệm và flashcards từ nội dung đã truy xuất, giúp người học ôn tập, kiểm tra kiến thức và ghi nhớ các ý chính.
4. **Cung cấp nhiều cách sử dụng:** Hệ thống có thể được truy cập qua giao diện web Streamlit, dòng lệnh CLI hoặc REST API bằng FastAPI.
5. **Đánh giá và tối ưu chất lượng RAG:** Project tích hợp đánh giá bằng Ragas để đo lường chất lượng truy xuất và sinh phản hồi, đồng thời thử nghiệm các chiến lược chunking và reranking.

I.2. Input và Output

Bài toán trong project tập trung vào việc xử lý tài liệu học tập và tạo ra các phản hồi có cấu trúc dựa trên nội dung tài liệu. Các thành phần đầu vào và đầu ra được mô tả như sau:

- **Input:** Các tệp PDF chứa nội dung học tập; câu hỏi hoặc yêu cầu dạng văn bản từ người dùng; các tùy chọn phạm vi xử lý như tên tài liệu, trang, bộ lọc metadata hoặc số lượng học liệu cần tạo.
- **Output:** Câu trả lời hỏi đáp có trích dẫn nguồn; bản tóm tắt và các ý chính; bộ câu hỏi trắc nghiệm; danh sách flashcards; kết quả có thể được hiển thị trên giao diện hoặc xuất ra JSON/Markdown.
- **Evaluation:** Bộ benchmark gồm các cặp câu hỏi và đáp án chuẩn được dùng để đánh giá. Kết quả đánh giá gồm các chỉ số Ragas như context recall, context precision, faithfulness và answer relevancy.

II. Nội dung project

Tổng quan, project được triển khai theo bốn nhóm nội dung chính: xây dựng kho tri thức từ tài liệu, cài đặt pipeline RAG, phát triển các chức năng học tập và giao diện sử dụng, sau đó đánh giá chất lượng hệ thống bằng các thí nghiệm định lượng.

II.1. Tiền xử lý và lưu trữ dữ liệu

Mục tiêu của giai đoạn này là chuyển đổi tài liệu PDF không cấu trúc thành các đơn vị tri thức có thể truy xuất được. Quy trình triển khai gồm:

1. **Nạp và trích xuất nội dung:** Hệ thống đọc các tệp PDF từ thư mục dữ liệu hoặc từ chức năng upload, sau đó trích xuất nội dung theo từng trang và gắn metadata nguồn như tên file, số trang và mã tài liệu.
2. **Phân đoạn văn bản:** Nội dung tài liệu được chia thành các chunk nhỏ bằng chiến lược chunking. Việc phân đoạn giúp hệ thống giữ được ngữ cảnh cục bộ, đồng thời phù hợp với giới hạn token của mô hình ngôn ngữ.
3. **Vector hóa và lưu trữ:** Mỗi chunk được chuyển thành embedding và lưu vào Qdrant cùng metadata nguồn. Nhờ đó, hệ thống có thể thực hiện tìm kiếm ngữ nghĩa thay vì chỉ khớp từ khóa.

II.2. Pipeline truy xuất và tạo sinh RAG

Đây là phần lõi của project, chịu trách nhiệm kết nối kho tri thức vector với mô hình ngôn ngữ. Pipeline RAG vận hành theo các bước chính:

1. **Truy xuất ngữ cảnh:** Khi nhận câu hỏi hoặc yêu cầu từ người dùng, hệ thống tìm các chunk liên quan nhất trong Qdrant bằng semantic search, có thể kết hợp với bộ lọc theo tài liệu, trang hoặc metadata.
2. **Đóng gói prompt:** Các chunk được truy xuất được đưa vào prompt template tương ứng với từng tác vụ như hỏi đáp, tóm tắt, tạo quiz hoặc tạo flashcards. Prompt được thiết kế để yêu cầu mô hình chỉ sử dụng thông tin có trong ngữ cảnh.
3. **Gọi mô hình ngôn ngữ:** Hệ thống hỗ trợ nhiều backend LLM như HuggingFace local, Gemini và vLLM. Các module còn lại chỉ gọi qua một hàm trung gian, giúp dễ thay đổi backend khi cần.
4. **Chuẩn hóa kết quả:** Kết quả sinh ra được đóng gói bằng các schema Pydantic, kèm trích dẫn nguồn và các chunk đã sử dụng làm ngữ cảnh.

II.3. Chức năng học tập và giao diện sử dụng

Project không chỉ dừng lại ở hỏi đáp, mà còn mở rộng thành một hệ thống hỗ trợ học tập dựa trên tài liệu. Các chức năng chính gồm:

1. **Hỏi đáp trên tài liệu:** Người dùng đặt câu hỏi, hệ thống truy xuất ngữ cảnh liên quan và sinh câu trả lời có dẫn chứng.
2. **Tóm tắt nội dung:** Hệ thống có thể tóm tắt theo toàn bộ tài liệu, theo bộ lọc hoặc theo một truy vấn cụ thể. Với tài liệu dài, hệ thống sử dụng chiến lược map-reduce để tổng hợp nội dung.
3. **Tạo Quiz và Flashcards:** Hệ thống sinh câu hỏi trắc nghiệm và thẻ ghi nhớ từ nội dung đã truy xuất. Output được validate để đảm bảo đúng cấu trúc và hạn chế item trùng lặp.
4. **Giao diện đa nền tảng:** Người dùng có thể tương tác với hệ thống qua Streamlit UI, CLI bằng Typer hoặc REST API bằng FastAPI. Kết quả học tập có thể xuất ra JSON hoặc Markdown để lưu trữ và chia sẻ.

II.4. Đánh giá và tối ưu hệ thống

Bên cạnh phần cài đặt, project còn có nhóm thực nghiệm nhằm đánh giá chất lượng RAG một cách định lượng. Phần này tập trung vào hai hướng chính:

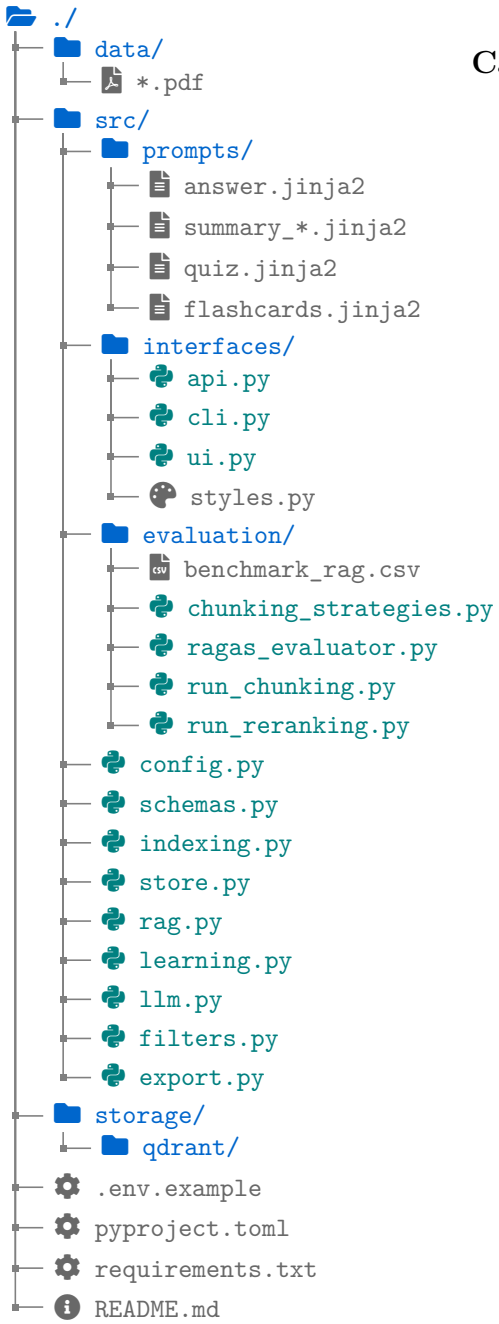
1. **Đánh giá bằng Ragas:** Hệ thống sử dụng bộ benchmark gồm các cặp câu hỏi và đáp án chuẩn để đo các chỉ số như context recall, context precision, faithfulness và answer relevancy. Các chỉ số này giúp phân tích chất lượng truy xuất và mức độ bám sát ngữ cảnh của câu trả lời.
2. **So sánh chiến lược Chunking:** Project thử nghiệm nhiều cấu hình chunking khác nhau, bao gồm recursive chunking và semantic chunking. Mỗi chiến lược được index vào một collection Qdrant riêng để so sánh công bằng trên cùng bộ benchmark.
3. **Đánh giá Reranking:** Hệ thống thử nghiệm thêm bước reranking bằng cross-encoder. Cách này truy xuất nhiều chunk ban đầu, chấm lại mức độ liên quan giữa câu hỏi và từng chunk, sau đó chỉ đưa các chunk tốt nhất vào prompt.

Nhờ cấu trúc này, project vừa cung cấp một ứng dụng NotebookLM đơn giản có thể sử dụng thực tế, vừa có cơ chế đánh giá để lựa chọn cấu hình retrieval phù hợp hơn cho tài liệu học tập.

III. Cài đặt chương trình

III.1. Cấu trúc mã nguồn của project

Trước khi đi vào chi tiết cài đặt, chúng ta cần nắm được cách tổ chức mã nguồn của project. Cấu trúc thư mục được thiết kế theo hướng tách rõ các nhóm chức năng chính. Cách tổ chức này giúp dễ theo dõi luồng xử lý từ lúc nạp tài liệu PDF, xây dựng kho tri thức, truy xuất ngữ cảnh, sinh phản hồi cho đến đánh giá chất lượng hệ thống.



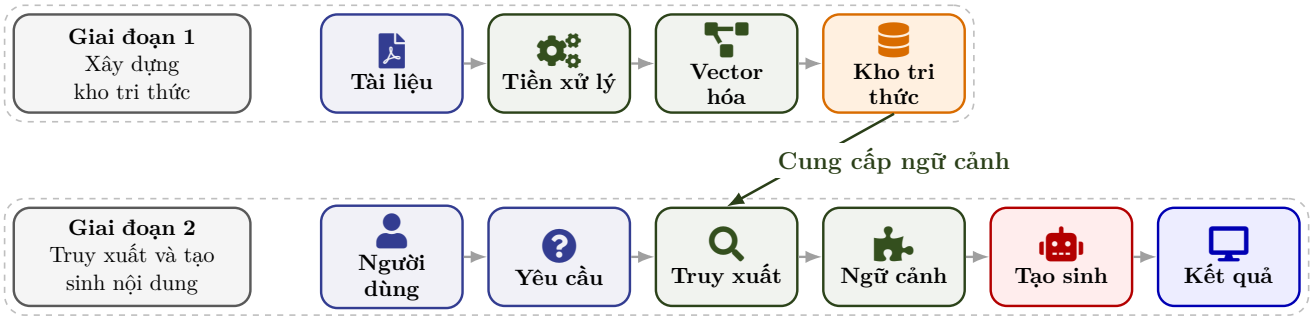
Các thành phần chính

- **Thư mục `data/`:** Chứa các tài liệu PDF đầu vào. Đây là nguồn dữ liệu để hệ thống đọc nội dung, chia đoạn và đưa vào kho vector.
- **Các file lõi trong `src/`:** Bao gồm `config.py`, `schemas.py`, `indexing.py`, `store.py`, `rag.py`, `learning.py`, `llm.py`, `filters.py` và `export.py`. Nhóm file này quản lý cấu hình, schema dữ liệu, nạp tài liệu, lưu trữ vector, truy xuất RAG, tạo học liệu, gọi LLM, lọc metadata và xuất kết quả.
- **Thư mục `src/prompts/`:** Lưu các prompt template dạng `Jinja2`. Mỗi template tương ứng với một tác vụ chính như hỏi đáp, tóm tắt, tạo quiz và tạo flashcards.
- **Thư mục `src/interfaces/`:** Cung cấp các lớp giao tiếp với hệ thống. `api.py` triển khai REST API bằng FastAPI, `cli.py` cung cấp giao diện dòng lệnh, `ui.py` xây dựng giao diện Streamlit, còn `styles.py` chứa CSS để định dạng giao diện web.
- **Thư mục `src/evaluation/`:** Chứa phần đánh giá chất lượng RAG.
- **Thư mục `storage/qdrant/`:** Lưu dữ liệu vector của Qdrant khi hệ thống chạy ở chế độ local. Thư mục này được tạo tự động trong quá trình index tài liệu.
- **Các file cấu hình project:** Gồm `.env.example`, `pyproject.toml`, `requirements.txt` và `README.md`, dùng để thiết lập môi trường, quản lý thư viện phụ thuộc và hướng dẫn sử dụng project.

Nhìn chung, project được tổ chức theo kiến trúc nhiều lớp. Các file lõi trong `src/` đảm nhiệm pipeline RAG và tạo học liệu; `interfaces/` cung cấp các cách sử dụng hệ thống; còn `evaluation/` tập trung đánh giá định lượng, gồm so sánh chiến lược chunking và kiểm tra hiệu quả của reranking.

III.2. Luồng hoạt động tổng thể

Hình 2 mô tả luồng hoạt động chính của hệ thống, từ bước xây dựng kho tri thức đến truy xuất ngữ cảnh và tạo sinh nội dung cho người dùng.



Hình 2: Luồng hoạt động tổng thể của project Learning System dựa trên kiến trúc RAG.

Sơ đồ 2 cho thấy hệ thống gồm hai giai đoạn chính: xây dựng kho tri thức từ tài liệu đầu vào và khai thác kho tri thức đó để truy xuất, tạo sinh nội dung và trả kết quả cho người dùng.

III.3. Cấu hình hệ thống và schema dữ liệu

Trước khi xử lý tài liệu hay gọi mô hình ngôn ngữ, hệ thống cần có một nơi quản lý thống nhất các tham số vận hành. Trong project này, phần cấu hình được đặt trong `src/config.py`. Module này dùng `BaseSettings` để khai báo giá trị mặc định, đồng thời cho phép ghi đè thông qua biến môi trường hoặc file `.env`.

src/config.py

```python
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_prefix="RAG_", extra="ignore
                                                ")
    data_dir: Path = Path("data")
    storage_dir: Path = Path("storage/qdrant")
    qdrant_collection: str = "rag_chunks"

    chunk_size: int = Field(default=1000, ge=100)
    chunk_overlap: int = Field(default=150, ge=0)
    top_k: int = Field(default=5, ge=1, le=64)

    embedding_model: str = "GreenNode/GreenNode-Embedding-Large-VN-Mixed-V1"

```

```
    llm_provider: Literal["hf_local", "gemini", "vllm"] = "hf_local"
    llm_temperature: float = Field(default=0.1, ge=0.0, le=2.0)

    hf_model: str = "/mnt/pretrained_fm/Qwen_Qwen3-4B-Instruct-2507"
    hf_device: int = 1
    hf_max_new_tokens: int = Field(default=2048, ge=1)

    gemini_model: str = "gemini-2.5-flash"
    google_api_key: str | None = Field(default=None, validation_alias="GOOGLE_API_KEY")

    vllm_api_base: str = "http://localhost:8001/v1"
    vllm_api_key: str = "EMPTY"

    summarize_batch_size: int = Field(default=10, ge=1)
    summarize_retrieval_k: int = Field(default=12, ge=1, le=128)
    generation_retrieval_k: int = Field(default=16, ge=1, le=128)

    quiz_default_count: int = Field(default=8, ge=1, le=50)
    flashcards_default_count: int = Field(default=15, ge=1, le=100)
    api_url: str = "http://localhost:8000"
```

Các nhóm tham số chính gồm:

- **Đường dẫn dữ liệu:** `data_dir` là nơi chứa PDF đầu vào, còn `storage_dir` là nơi lưu dữ liệu vector của Qdrant khi chạy local.
- **Tham số chunking và retrieval:** `chunk_size`, `chunk_overlap` quyết định cách chia nhỏ tài liệu; `top_k` là số chunk mặc định được truy xuất khi hỏi đáp.
- **Mô hình embedding:** `embedding_model` xác định mô hình dùng để chuyển văn bản thành vector.
- **Mô hình sinh ngôn ngữ:** `llm_provider` cho phép chọn một trong ba backend: HuggingFace local, Gemini hoặc vLLM.
- **Tham số học tập:** `summarize_batch_size`, `summarize_retrieval_k`, `generation_retrieval_k`, `quiz_default_count` và `flashcards_default_count` điều khiển phạm vi ngữ cảnh và số lượng học liệu được tạo.

Cấu hình cũng có bước kiểm tra hợp lệ bằng `model_validator`. Ví dụ, `chunk_overlap` phải nhỏ hơn `chunk_size`, hoặc khi dùng Gemini thì cần có `GOOGLE_API_KEY`.

src/config.py

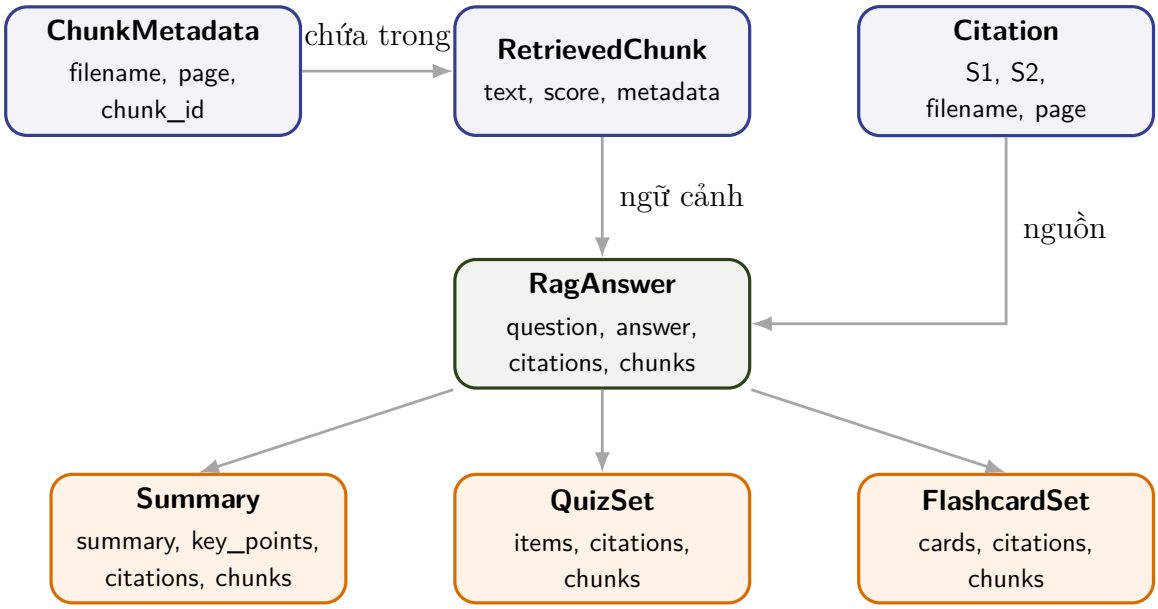
```
@model_validator(mode="after")
def validate_config(self) -> "Settings":
    if self.chunk_overlap >= self.chunk_size:
        raise ValueError("chunk_overlap must be smaller than chunk_size.")
    if self.hf_device < -1:
        raise ValueError("hf_device must be -1 for CPU or >= 0 for CUDA.")
    if self.llm_provider == "gemini" and not self.google_api_key:
```

```
        raise ValueError("GOOGLE_API_KEY is required when llm_provider='gemini'.")
    return self

@lru_cache(maxsize=1)
def get_settings() -> Settings:
    return Settings()

settings = get_settings()
```

Song song với cấu hình, project định nghĩa các kiểu dữ liệu chuẩn trong `src/schemas.py`. Các schema này giúp dữ liệu truyền giữa các module luôn có cấu trúc thống nhất, đặc biệt khi kết quả cần trả về qua API, CLI hoặc giao diện web. Hình 3 minh họa mối quan hệ các schema trong project.



Hình 3: Quan hệ giữa các schema dữ liệu chính trong hệ thống.

src/schemas.py

```
class ChunkMetadata(BaseModel):
    document_id: str
    filename: str
    source: str
    page: int
    chunk_id: str
    section: str | None = None

class RetrievedChunk(BaseModel):
    text: str
    score: float
    metadata: ChunkMetadata

```

```python
class Citation(BaseModel):
    source_index: int
    source_marker: str
    filename: str
    page: int
    section: str | None = None
    chunk_id: str | None = None

class RagAnswer(BaseModel):
    question: str
    answer: str
    citations: list[Citation] = Field(default_factory=list)
    chunks: list[RetrievedChunk] = Field(default_factory=list)
```

Có thể hiểu các schema trên như sau:

- **`ChunkMetadata`:** Lưu thông tin nguồn của mỗi chunk, gồm tài liệu, tên file, trang, đường dẫn nguồn và `chunk_id`.
- **`RetrievedChunk`:** Biểu diễn một đoạn văn bản được truy xuất, gồm nội dung, điểm tương đồng và metadata.
- **`Citation`:** Chuẩn hóa nguồn trích dẫn theo các marker như `S1`, `S2`.
- **`RagAnswer`:** Là kết quả cuối cùng của chức năng hỏi đáp, gồm câu hỏi, câu trả lời, danh sách trích dẫn và các chunk nguồn.

Với các chức năng học tập, hệ thống định nghĩa thêm `Summary`, `QuizSet` và `FlashcardSet`. Điểm cập nhật quan trọng là các output này lưu thêm `chunks`, giúp giao diện và quá trình debug có thể hiển thị lại ngữ cảnh đã dùng để tạo nội dung.

src/schemas.py

```python
class Summary(BaseModel):
    scope: Literal["query", "document", "filter", "corpus"]
    target: str | None = None
    summary: str
    key_points: list[str] = Field(default_factory=list)
    citations: list[Citation] = Field(default_factory=list)
    chunks: list[RetrievedChunk] = Field(default_factory=list)

class QuizItem(BaseModel):
    question: str
    options: list[str] = Field(min_length=4, max_length=4)
    correct_index: int
    explanation: str
    source_markers: list[str] = Field(default_factory=list)
    difficulty: str | None = None
    topic: str | None = None

    @model_validator(mode="after")
```

```python
    def _validate_correct_index(self) -> "QuizItem":
        if not 0 <= self.correct_index < len(self.options):
            raise ValueError("correct_index out of range")
        return self

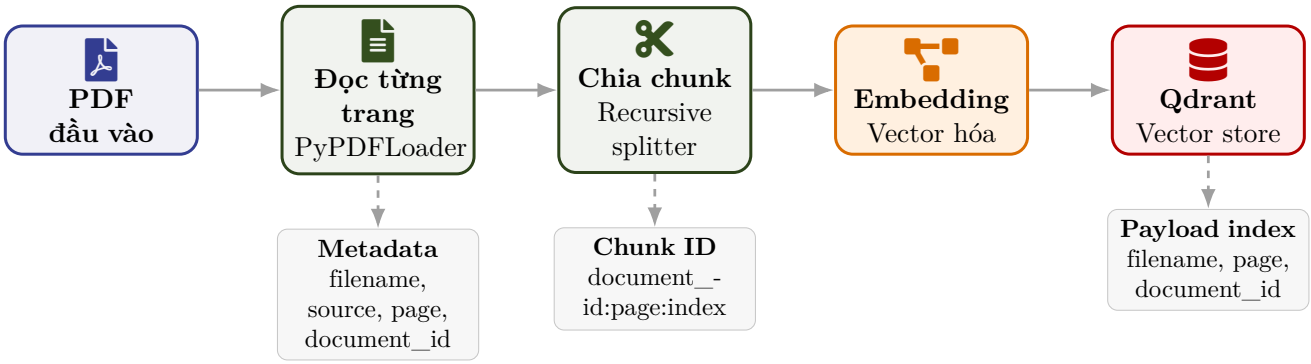
class QuizSet(BaseModel):
    scope: Literal["query", "document", "filter", "corpus"]
    target: str | None = None
    items: list[QuizItem] = Field(default_factory=list)
    citations: list[Citation] = Field(default_factory=list)
    chunks: list[RetrievedChunk] = Field(default_factory=list)

class Flashcard(BaseModel):
    front: str
    back: str
    hint: str | None = None
    topic: str | None = None
    source_markers: list[str] = Field(default_factory=list)

class FlashcardSet(BaseModel):
    scope: Literal["query", "document", "filter", "corpus"]
    target: str | None = None
    cards: list[Flashcard] = Field(default_factory=list)
    citations: list[Citation] = Field(default_factory=list)
    chunks: list[RetrievedChunk] = Field(default_factory=list)
```

III.4. Nạp tài liệu, chia chunk và lưu trữ vector

Giai đoạn đầu tiên của hệ thống là đưa tài liệu PDF vào kho tri thức. Luồng xử lý chính nằm trong `src/indexing.py`, gồm các bước như Hình 4: tìm file PDF, đọc nội dung từng trang, chia nhỏ văn bản thành chunk, gắn metadata và lưu các chunk vào Qdrant.



Hình 4: Pipeline nạp tài liệu, chia chunk, vector hóa và lưu trữ vào Qdrant.

Trước hết, hệ thống dùng `PyPDFLoader` để đọc từng file PDF. Mỗi trang sau khi đọc được gắn metadata để giữ lại thông tin nguồn.

src/indexing.py

```python
def _document_id(path):
    raw = f"{path.name}:{path.stat().st_size}"
    return hashlib.sha1(raw.encode("utf-8")).hexdigest()[:16]

def _chunk_id(doc_id, page, index):
    return f"{doc_id}:{page}:{index}"

def _load_pdf(path):
    pages = PyPDFLoader(str(path)).load()
    doc_id = _document_id(path)

    for doc in pages:
        page_number = int(doc.metadata.get("page", 0)) + 1
        doc.metadata = {
            "document_id": doc_id,
            "filename": path.name,
            "source": str(path.resolve()),
            "page": page_number,
            "section": doc.metadata.get("section"),
        }
    return pages
```

Trong đó, `_document_id` tạo mã định danh cho tài liệu, `_chunk_id` tạo mã cho từng chunk, còn `_load_pdf` chuẩn hóa metadata theo từng trang để các bước truy xuất và trích dẫn sau này có nguồn rõ ràng.

Sau khi có dữ liệu theo trang, hệ thống dùng `RecursiveCharacterTextSplitter` để chia nhỏ văn bản. Bộ tách này ưu tiên chia theo đoạn, dòng, câu, khoảng trắng rồi mới đến ký tự đơn lẻ, giúp chunk giữ được ngữ cảnh tự nhiên hơn.

src/indexing.py

```python
def _splitter(chunk_size=None, chunk_overlap=None):
    return RecursiveCharacterTextSplitter(
        chunk_size=chunk_size or settings.chunk_size,
        chunk_overlap=chunk_overlap or settings.chunk_overlap,
        separators=["\n\n", "\n", ". ", " ", ""],
        keep_separator=False,
    )
```

Hàm `build_chunks` kết hợp bước đọc PDF và chia chunk. Hàm này cũng hỗ trợ truyền vào `chunker` tùy chỉnh, nhờ đó phần evaluation có thể thử nhiều chiến lược chunking khác nhau mà không cần thay đổi logic index.

src/indexing.py

```python
def build_chunks(pdf_paths, chunk_size=None, chunk_overlap=None, chunker=None):
    page_docs = []
```

```
    for path in pdf_paths:
        page_docs.extend(_load_pdf(path))

    splitter = chunker or _splitter(chunk_size, chunk_overlap)
    chunks = splitter.split_documents(page_docs)
    per_doc_counter = defaultdict(int)

    for chunk in chunks:
        doc_id = chunk.metadata["document_id"]
        idx = per_doc_counter[doc_id]
        per_doc_counter[doc_id] += 1

        meta = ChunkMetadata(
            document_id=doc_id,
            filename=chunk.metadata["filename"],
            source=chunk.metadata["source"],
            page=chunk.metadata["page"],
            chunk_id=_chunk_id(doc_id, chunk.metadata["page"], idx),
            section=chunk.metadata.get("section"),
        )
        chunk.metadata = meta.model_dump()

    return chunks
```

Phần lưu trữ vector được quản lý trong `src/store.py`. Project sử dụng `HuggingFaceEmbeddings` để tạo vector và `QdrantClient` để lưu trữ vector ở chế độ local.

src/store.py

```
@lru_cache(maxsize=1)
def get_embeddings():
    return HuggingFaceEmbeddings(
        model_name=settings.embedding_model,
        model_kwargs={"device": settings.hf_device},
        encode_kwargs={"normalize_embeddings": True},
    )

@lru_cache(maxsize=1)
def get_client():
    settings.storage_dir.mkdir(parents=True, exist_ok=True)
    return QdrantClient(path=str(settings.storage_dir))

def get_vector_store(collection_name=None):
    return QdrantVectorStore(
        client=get_client(),
        collection_name=collection_name or settings.qdrant_collection,
        embedding=get_embeddings(),
    )
```

Các hàm trên dùng `@lru_cache` để tránh khởi tạo lại mô hình embedding và Qdrant client nhiều lần. Đây là chi tiết quan trọng vì mô hình embedding thường tốn tài nguyên khi tải lên GPU

hoặc CPU.

Trước khi ghi dữ liệu, hệ thống đảm bảo collection trong Qdrant đã tồn tại. Nếu chưa có, collection được tạo mới với số chiều vector tương ứng với mô hình embedding. Các trường metadata quan trọng cũng được tạo payload index để hỗ trợ lọc theo tài liệu hoặc trang.

src/store.py

```python
INDEXED_PAYLOAD_FIELDS = {
    "metadata.document_id": qmodels.PayloadSchemaType.KEYWORD,
    "metadata.filename": qmodels.PayloadSchemaType.KEYWORD,
    "metadata.page": qmodels.PayloadSchemaType.INTEGER,
}

def ensure_collection(recreate=False, collection_name=None):
    client = get_client()
    name = collection_name or settings.qdrant_collection
    exists = client.collection_exists(name)

    if exists and recreate:
        client.delete_collection(name)
        exists = False

    if not exists:
        dim = len(get_embeddings().embed_query("dimension probe"))
        client.create_collection(
            collection_name=name,
            vectors_config=qmodels.VectorParams(size=dim, distance=qmodels.Distance.
                                                COSINE),
        )

    payload_schema = client.get_collection(name).payload_schema or {}
    for field, schema in INDEXED_PAYLOAD_FIELDS.items():
        if payload_schema.get(field) is None:
            client.create_payload_index(name, field_name=field, field_schema=schema)
```

Sau khi collection đã sẵn sàng, các chunk được ghi vào vector store bằng `index_chunks`. Mỗi chunk được gán UUID ổn định từ `chunk_id`, giúp việc nạp lại cùng một tài liệu tránh tạo bản ghi trùng lặp.

src/indexing.py

```python
def index_chunks(chunks, collection_name=None):
    if not chunks:
        return 0

    ids = [str(uuid.uuid5(uuid.NAMESPACE_DNS, c.metadata["chunk_id"])) for c in chunks]
    get_vector_store(collection_name=collection_name).add_documents(chunks, ids=ids)
    return len(chunks)

def ingest(recreate=False, collection_name=None, chunker=None, chunk_size=None,
```

```
                                                chunk_overlap=None):
    pdfs = discover_pdfs()
    ensure_collection(recreate=recreate, collection_name=collection_name)
    chunks = build_chunks(pdfs, chunker=chunker, chunk_size=chunk_size, chunk_overlap=
                                                chunk_overlap)
    return index_chunks(chunks, collection_name=collection_name)

def save_and_ingest_pdf(file_bytes, filename):
    safe_name = Path(filename).name
    dest = settings.data_dir / safe_name
    settings.data_dir.mkdir(parents=True, exist_ok=True)
    dest.write_bytes(file_bytes)

    ensure_collection(recreate=False)
    chunks = build_chunks([dest])
    return {"filename": safe_name, "chunks_indexed": index_chunks(chunks)}
```

Ngoài chế độ nạp toàn bộ PDF trong thư mục `data/`, hệ thống còn có `save_and_ingest_pdf` để phục vụ chức năng upload từ giao diện web hoặc API.

III.5. Chuẩn hóa bộ lọc và truy xuất ngữ cảnh

Khi người dùng hỏi đáp hoặc tạo học liệu, hệ thống thường không cần truy xuất toàn bộ kho tài liệu. Người dùng có thể chọn một file, nhiều file hoặc một trang cụ thể. Để xử lý thống nhất các trường hợp này, project dùng module `src/filters.py`.

src/filters.py

```
class MetadataFilter(BaseModel):
    filename: str | None = None
    filenames: list[str] | None = None
    page: int | None = None
    section: str | None = None
    document_id: str | None = None

    @model_validator(mode="after")
    def _normalize(self) -> "MetadataFilter":
        names = [n.strip() for n in (self.filenames or []) if isinstance(n, str) and n.
                                                strip()]
        if not names:
            self.filenames = None
        elif len(names) == 1:
            self.filename, self.filenames = names[0], None
        else:
            self.filename, self.filenames, self.page = None, names, None

        if self.filename is not None:
            self.filename = self.filename.strip() or None
        if self.section is not None:
            self.section = self.section.strip() or None
```

```
        if self.document_id is not None:
            self.document_id = self.document_id.strip() or None

        return self
```

Logic chuẩn hóa filter giúp hệ thống xử lý dữ liệu đầu vào an toàn hơn: nếu chỉ có một file thì chuyển về `filename`; nếu có nhiều file thì dùng `filenames` và bỏ `page` để tránh nhập nhằng giữa các tài liệu; các trường dạng chuỗi cũng được loại bỏ khoảng trắng thừa.

Sau khi được chuẩn hóa, filter được chuyển sang định dạng của Qdrant. Đây là bước nối giữa dữ liệu đầu vào từ UI/API/CLI và truy vấn thực tế trong vector database.

src/filters.py

```python
def filters_to_dict(filters):
    f = _coerce_filter(filters)
    return None if f is None else f.model_dump(exclude_none=True) or None

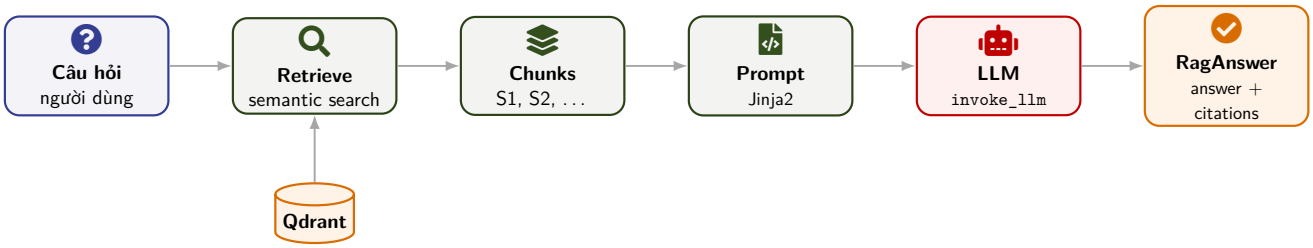
def filters_to_qdrant(filters):
    flat = filters_to_dict(filters)
    if not flat:
        return None

    conditions = []
    for field, value in flat.items():
        if field == "filenames" and isinstance(value, list):
            conditions.append(qmodels.FieldCondition(
                key="metadata.filename", match=qmodels.MatchAny(any=value)
            ))
        elif isinstance(value, (str, int)):
            conditions.append(qmodels.FieldCondition(
                key=f"metadata.{field}", match=qmodels.MatchValue(value=value)
            ))

    return qmodels.Filter(must=conditions) if conditions else None
```

III.6. Cài đặt pipeline RAG

Phần lõi của hệ thống RAG nằm trong `src/rag.py`. Được minh họa trong Hình 5, module này chịu trách nhiệm truy xuất ngữ cảnh, render prompt, gọi LLM và đóng gói câu trả lời kèm trích dẫn. Khi người dùng đặt câu hỏi, hàm `retrieve` thực hiện semantic search trên Qdrant để tìm các chunk liên quan nhất.



Hình 5: Pipeline RAG chi tiết trong chức năng hỏi đáp trên tài liệu.

src/rag.py

```python
def retrieve(query, k=None, filters=None, collection_name=None):
    hits = get_vector_store(collection_name).similarity_search_with_score(
        query=query,
        k=k or settings.top_k,
        filter=filters_to_qdrant(filters),
    )

    return [
        RetrievedChunk(text=doc.page_content, score=float(score), metadata=
                                            ChunkMetadata(**doc.metadata))
        for doc, score in hits
    ]
```

Kết quả trả về là danh sách `RetrievedChunk`. Mỗi phần tử gồm nội dung chunk, điểm tương đồng và metadata nguồn dùng cho trích dẫn.

Ngoài truy xuất theo semantic search, hệ thống còn có hàm `fetch_all_chunks`. Hàm này dùng cơ chế `scroll` của Qdrant để lấy toàn bộ chunk thỏa mãn filter. Đây là lựa chọn phù hợp cho tóm tắt toàn bộ tài liệu hoặc tạo học liệu từ một phạm vi cố định.

src/rag.py

```python
def fetch_all_chunks(filters=None, collection_name=None):
    name = collection_name or settings.qdrant_collection
    results = []

    for page in scroll_all(name, scroll_filter=filters_to_qdrant(filters)):
        for point in page:
            payload = point.payload or {}
            meta, text = payload.get("metadata") or {}, payload.get("page_content") or
                                                ""
            if meta and text:
                results.append(RetrievedChunk(text=text, score=0.0, metadata=
                                            ChunkMetadata(**meta)))

    return sorted(results, key=lambda r: (
        r.metadata.filename,
        r.metadata.page,
        int(r.metadata.chunk_id.rsplit(":", 1)[-1]),
    ))
```

Sau khi có ngữ cảnh, hệ thống dùng `Jinja2` để render prompt. Các prompt được tách riêng trong thư mục `src/prompts/`, giúp dễ chỉnh hành vi của LLM mà không cần thay đổi logic Python.

src/rag.py

```python
@lru_cache(maxsize=1)
def _jinja_env():
    return Environment(
        loader=FileSystemLoader(str(PROMPTS_DIR)),
        autoescape=False, undefined=StrictUndefined,
        trim_blocks=True, lstrip_blocks=True,
    )

def render_prompt(template_name, **context):
    return _jinja_env().get_template(template_name).render(**context)
```

Cuối cùng, hàm `answer` kết hợp toàn bộ pipeline hỏi đáp: truy xuất chunk, render prompt, gọi LLM, rồi trả về câu trả lời có nguồn trích dẫn.

src/rag.py

```python
def format_citations(chunks):
    return [
        Citation(
            source_index=i,
            source_marker=f"S{i}",
            filename=c.metadata.filename,
            page=c.metadata.page,
            section=c.metadata.section,
            chunk_id=c.metadata.chunk_id,
        )
        for i, c in enumerate(chunks, start=1)
    ]

def answer(question, k=None, filters=None, collection_name=None):
    chunks = retrieve(question, k=k, filters=filters, collection_name=collection_name)

    if not chunks:
        return RagAnswer(
            question=question,
            answer="Tôi không có đủ thông tin trong ngữ cảnh được cung cấp để trả lời."
                                                )

    prompt = render_prompt(ANSWER_TEMPLATE, question=question, chunks=chunks)
    text = invoke_llm(prompt)

    return RagAnswer(
        question=question,
        answer=text.strip(),
```

```
        citations=format_citations(chunks),
        chunks=chunks,
    )
```

Tóm lại, luồng hỏi đáp RAG gồm năm bước chính: nhận câu hỏi, truy xuất ngữ cảnh, đóng gói prompt, gọi LLM và trả về `RagAnswer` kèm trích dẫn nguồn.

III.7. Tích hợp các backend LLM

Module `src/llm.py` cung cấp một lớp trung gian để gọi mô hình ngôn ngữ. Nhờ module này, các phần còn lại của hệ thống không cần quan tâm mô hình đang chạy bằng HuggingFace local, Gemini hay vLLM. Tất cả chỉ cần gọi chung hàm `invoke_llm`.

src/llm.py

```python
def _build_hf_local():
    import torch
    from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline
    from langchain_huggingface import ChatHuggingFace, HuggingFacePipeline

    tokenizer = AutoTokenizer.from_pretrained(settings.hf_model)
    model = AutoModelForCausalLM.from_pretrained(settings.hf_model, dtype=torch.
                                                 bfloat16)

    text_gen = pipeline(
        task="text-generation",
        model=model,
        tokenizer=tokenizer,
        device=settings.hf_device,
        return_full_text=False,
    )
    text_gen.generation_config.max_new_tokens = settings.hf_max_new_tokens
    text_gen.generation_config.do_sample = settings.llm_temperature > 0

    return ChatHuggingFace(llm=HuggingFacePipeline(pipeline=text_gen))
```

Với chế độ HuggingFace local, hệ thống tải tokenizer, model và tạo pipeline sinh văn bản. Các tham số như số token sinh tối đa, nhiệt độ và thiết bị chạy được lấy từ `settings`.

Hai backend còn lại là Gemini và vLLM. Gemini được gọi qua `ChatGoogleGenerativeAI`, còn vLLM được gọi thông qua API tương thích OpenAI bằng `ChatOpenAI`.

src/llm.py

```python
def _build_gemini():
    return ChatGoogleGenerativeAI(
        model=settings.gemini_model,
```

```
        temperature=settings.llm_temperature,
        google_api_key=settings.google_api_key,
    )

def _build_vllm():
    return ChatOpenAI(
        model=settings.hf_model,
        openai_api_key=settings.vllm_api_key,
        openai_api_base=settings.vllm_api_base,
        temperature=settings.llm_temperature,
    )
```

Hàm `get_llm` chọn backend dựa trên `settings.llm_provider`. Kết quả được cache để tránh khởi tạo lại mô hình nhiều lần.

src/llm.py

```
@lru_cache(maxsize=4)
def get_llm(provider=None):
    provider = provider or settings.llm_provider

    if provider == "hf_local":
        return _build_hf_local()
    if provider == "gemini":
        return _build_gemini()
    if provider == "vllm":
        return _build_vllm()

    raise ValueError(f"Unknown llm_provider '{provider}'")

def invoke_llm(prompt, provider=None):
    response = get_llm(provider=provider).invoke([HumanMessage(content=prompt)])
    return response.content if isinstance(response.content, str) else str(response.
                                                content)
```

Thiết kế này giúp hệ thống dễ mở rộng. Nếu muốn đổi backend sinh ngôn ngữ, ta chỉ cần thay `llm_provider`, trong khi các module như `rag.py` và `learning.py` vẫn giữ nguyên cách gọi.

III.8. Các chức năng học tập: tóm tắt, quiz và flashcards

Các chức năng học tập được cài đặt trong `src/learning.py`. Điểm chung của cả ba chức năng là đều bám sát nội dung đã index, không sinh học liệu từ tri thức bên ngoài một cách tự do. Trước khi gọi LLM, hệ thống luôn xác định phạm vi nội dung cần xử lý.

src/learning.py

```python
def _resolve_target(document, query, filters, k, retrieval_k):
    effective_filters = dict(filters or {})

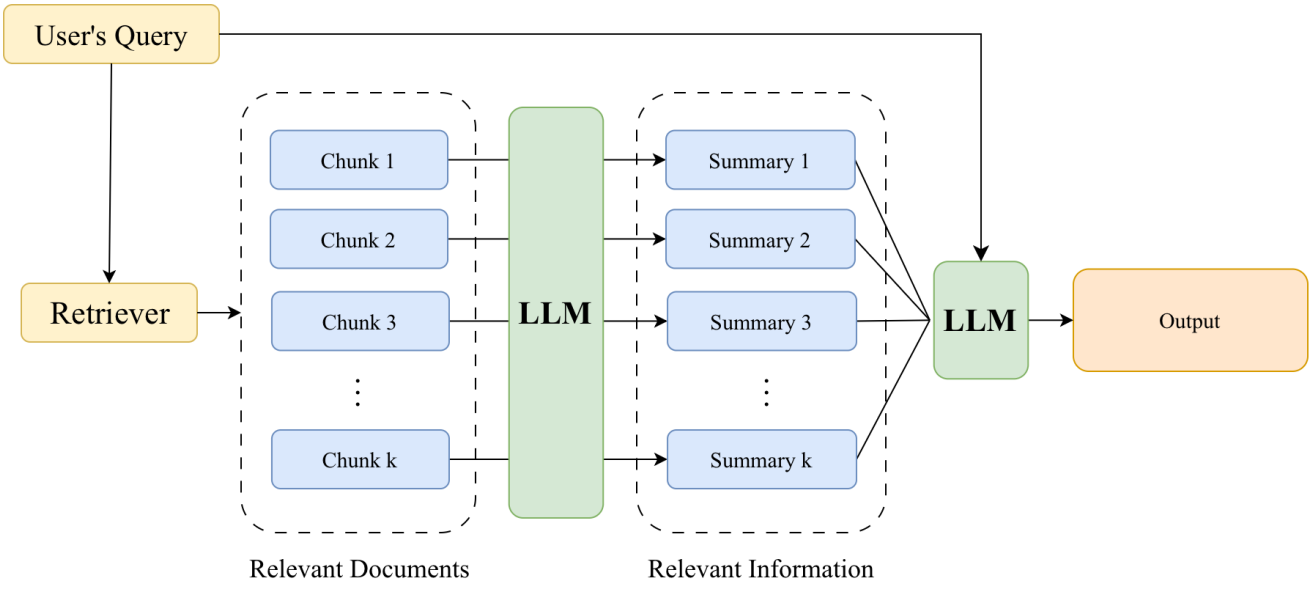
    if document:
        effective_filters["filename"] = document

    if query:
        chunks = retrieve(query, k=k or retrieval_k, filters=effective_filters)
        return chunks, "query", query

    if effective_filters:
        chunks = fetch_all_chunks(filters=effective_filters)
        scope = "document" if document else "filter"
        target = ", ".join(f"{k}={v}" for k, v in effective_filters.items())
        return chunks, scope, target

    return fetch_all_chunks(filters=None), "corpus", None
```

Hàm `_resolve_target` là điểm điều phối quan trọng: nếu có `query` thì dùng semantic search; nếu có `document` hoặc `filters` thì lấy toàn bộ chunk trong phạm vi đó; nếu không có điều kiện nào thì xử lý toàn bộ corpus.



Hình 6: Chiến lược Map-Reduce trong tóm tắt văn bản dài.

Đối với chức năng tóm tắt, hệ thống hỗ trợ hai chế độ. Nếu số chunk không quá lớn, hệ thống tóm tắt một lần bằng prompt `summary_single.jinja2`. Nếu tài liệu dài, hệ thống dùng chiến lược **Map-Reduce**: tóm tắt từng nhóm chunk trước, sau đó tổng hợp các bản tóm tắt con thành bản tóm tắt cuối cùng (Hình 6).

src/learning.py

```python
def summarize(document=None, query=None, filters=None, k=None):
    chunks, scope, target = _resolve_target(
        document, query, filters, k, settings.summarize_retrieval_k
    )

    if len(chunks) <= settings.summarize_batch_size:
        prompt = render_prompt(SUMMARY_SINGLE_TEMPLATE, chunks=chunks)
        payload = _parse_json(invoke_llm(prompt))
        summary_text, key_points = _validate_summary_payload(payload)
    else:
        partials = []
        for start in range(0, len(chunks), settings.summarize_batch_size):
            batch = chunks[start : start + settings.summarize_batch_size]
            payload = _parse_json(invoke_llm(render_prompt(SUMMARY_MAP_TEMPLATE, chunks
                                                =batch)))
            summary_text, key_points = _validate_summary_payload(payload)
            partials.append({"summary": summary_text, "key_points": key_points})

        payload = _parse_json(invoke_llm(render_prompt(SUMMARY_REDUCE_TEMPLATE,
                                            partials=partials)))
        summary_text, key_points = _validate_summary_payload(payload)

    return Summary(
        scope=scope,
        target=target,
        summary=summary_text,
        key_points=key_points,
        citations=format_citations(chunks),
        chunks=chunks,
    )
```

Với quiz và flashcards, hệ thống yêu cầu LLM trả về JSON. Sau đó JSON được parse và validate bằng các schema Pydantic. Cách làm này giúp loại bỏ item không hợp lệ, tránh trùng lặp và đảm bảo output có cấu trúc rõ ràng để hiển thị trên giao diện.

src/learning.py

```python
def _parse_json(text):
    cleaned = text.strip()
    if cleaned.startswith("```"):
        cleaned = cleaned.split("\n", 1)[-1].removesuffix("```").strip()
    obj = json.loads(cleaned)

    if not isinstance(obj, (dict, list)):
        raise RuntimeError("Expected JSON object or array.")
    return obj

def _validate_items(payload, key, model_class, dedup_field, label, valid_markers):
    raw_items = payload.get(key)
    items, seen = [], set()
```

```python

    for raw in raw_items:
        try:
            item = model_class.model_validate(raw)
        except ValidationError:
            continue

        norm = str(getattr(item, dedup_field, "")).strip().lower()
        if not norm or norm in seen:
            continue

        seen.add(norm)
        markers = [m for m in item.source_markers if m in valid_markers]
        items.append(item.model_copy(update={"source_markers": markers}))

    if not items:
        raise RuntimeError(f"No valid {label} produced.")
    return items
```

Chức năng tạo quiz dùng `QuizItem` để đảm bảo mỗi câu hỏi có bốn lựa chọn, đáp án đúng, giải thích và nguồn tham chiếu.

src/learning.py

```python
def generate_quiz(document=None, query=None, filters=None, count=None, k=None):
    chunks, scope, target = _resolve_target(
        document, query, filters, k, settings.generation_retrieval_k
    )

    n = count or settings.quiz_default_count
    valid_markers = {f"S{i}" for i in range(1, len(chunks) + 1)}
    prompt = render_prompt(QUIZ_TEMPLATE, chunks=chunks, count=n)
    payload = _parse_json(invoke_llm(prompt))

    items = _validate_items(payload, "items", QuizItem, "question", "quiz items",
                                            valid_markers)

    return QuizSet(scope=scope, target=target, items=items, chunks=chunks,
        citations=format_citations(chunks))
```

Chức năng tạo flashcards có luồng xử lý tương tự, nhưng schema đầu ra là `Flashcard`. Mỗi thẻ gồm mặt trước, mặt sau, gợi ý nếu có, chủ đề và nguồn tham chiếu.

src/learning.py

```python
def generate_flashcards(document=None, query=None, filters=None, count=None, k=None):
    chunks, scope, target = _resolve_target(
        document, query, filters, k, settings.generation_retrieval_k
    )

```

```python
    n = count or settings.flashcards_default_count
    valid_markers = {f"S{i}" for i in range(1, len(chunks) + 1)}
    prompt = render_prompt(FLASHCARDS_TEMPLATE, chunks=chunks, count=n)
    payload = _parse_json(invoke_llm(prompt))

    cards = _validate_items(payload, "cards", Flashcard, "front", "flashcards",
                                            valid_markers)

    return FlashcardSet(scope=scope,
        target=target, cards=cards, chunks=chunks,
        citations=format_citations(chunks))
```

Nhìn chung, ba chức năng học tập dùng chung một nguyên tắc: xác định phạm vi nội dung, lấy chunk phù hợp, đưa chunk vào prompt, yêu cầu LLM trả về dữ liệu có cấu trúc, sau đó validate và gắn trích dẫn nguồn.

III.9. Xuất kết quả học tập

Sau khi tạo tóm tắt, quiz hoặc flashcards, người dùng có thể cần lưu kết quả lại để học hoặc chia sẻ. Module `src/export.py` đảm nhiệm việc chuyển các schema Pydantic sang JSON hoặc Markdown.

src/export.py

```python
ExportFormat = Literal["text", "md", "json"]

def export(model, *, fmt="text", output=None):
    if fmt == "json":
        text = model.model_dump_json(indent=2) + "\n"
    elif fmt in {"text", "md"}:
        text = _to_markdown(model)
    else:
        raise ValueError(f"Unknown fmt '{fmt}'. Expected 'text' | 'md' | 'json'.")

    if output is None:
        return text

    output.parent.mkdir(parents=True, exist_ok=True)
    output.write_text(text, encoding="utf-8")
    return output
```

Cách thiết kế này giúp CLI và giao diện web dùng chung một logic xuất file. Nếu `output` không được truyền vào, hàm trả về chuỗi kết quả. Nếu có `output`, hàm ghi nội dung ra file và trả về đường dẫn đã ghi.

III.10. API, CLI và giao diện người dùng

Project cung cấp nhiều cách tương tác với hệ thống, nhưng các giao diện này chỉ đóng vai trò lớp bên ngoài. Logic cốt lõi vẫn nằm ở các module `indexing.py`, `rag.py`, `learning.py` và `export.py`.

Với REST API, file `src/interfaces/api.py` dùng FastAPI để định nghĩa các endpoint chính. API nhận request, chuẩn hóa input, gọi các hàm lõi và trả kết quả theo schema đã định nghĩa.

src/interfaces/api.py

```python
class AskRequest(BaseModel):
    question: str = Field(min_length=1)
    k: int | None = Field(default=None, ge=1, le=64)
    filters: MetadataFilter | None = None

class SummarizeRequest(BaseModel):
    document: str | None = None
    query: str | None = None
    filters: MetadataFilter | None = None
    k: int | None = Field(default=None, ge=1, le=64)

class QuizRequest(BaseModel):
    document: str | None = None
    query: str | None = None
    filters: MetadataFilter | None = None
    count: int | None = Field(default=None, ge=1, le=50)
    k: int | None = Field(default=None, ge=1, le=64)

class FlashcardsRequest(QuizRequest):
    pass
```

Các request schema trên phản ánh đúng cách hệ thống hoạt động: hỏi đáp cần câu hỏi và số chunk truy xuất; tóm tắt, quiz và flashcards có thể chạy theo tài liệu, truy vấn hoặc bộ lọc; riêng quiz và flashcards có thêm `count` để điều chỉnh số lượng học liệu.

src/interfaces/api.py

```python
app = FastAPI(
    title="RAG Learning API",
    description="Grounded Q&A, summaries, quizzes, and flashcards over indexed PDFs.",
    version="0.1.0",
)

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/documents", response_model=list[DocumentInfo])
def documents():
    return list_documents()
```

```python

@app.post("/upload", response_model=UploadResponse)
async def upload(file: UploadFile = File(...)):
    content = await file.read()
    return save_and_ingest_pdf(content, file.filename or "")

@app.post("/ask", response_model=RagAnswer)
def ask(req: AskRequest):
    return answer(req.question, k=req.k, filters=filters_to_dict(req.filters))

@app.post("/summarize", response_model=Summary)
def summarize(req: SummarizeRequest):
    return summarize_learning(
        document=req.document,
        query=req.query,
        filters=filters_to_dict(req.filters),
        k=req.k,
    )
```

Các endpoint chính gồm:

- **`/health`:** Kiểm tra trạng thái hoạt động của API.
- **`/upload`:** Nhận PDF, lưu vào thư mục `data/` và index vào Qdrant.
- **`/documents`:** Liệt kê các tài liệu đã được index, bao gồm tên file, mã tài liệu, số trang và số chunk.
- **`/ask`:** Thực hiện hỏi đáp RAG có trích dẫn.
- **`/summarize`:** Tạo tóm tắt dựa trên tài liệu, truy vấn hoặc filter.
- **`/quiz`:** Tạo câu hỏi trắc nghiệm từ nội dung đã index.
- **`/flashcards`:** Tạo flashcards phục vụ ôn tập.

Với giao diện dòng lệnh, project dùng Typer trong `src/interfaces/cli.py`. CLI phù hợp cho việc kiểm thử nhanh hoặc chạy các chức năng trong môi trường terminal.

src/interfaces/cli.py

```python
@app.command()
def ingest(recreate=False):
    count = ingest_data_dir(recreate=recreate)
    typer.echo(f"Done. {count} chunks indexed.")

@app.command()
def ask(question, k=None, filters=None):
    result = answer(question, k=k, filters=_parse_filters(filters))
    _print_answer(result.answer)
    _print_sources(result.chunks)
```

```python

@app.command("debug-retrieval")
def debug_retrieval(question, k=None, filters=None, as_json=False):
    chunks = retrieve(question, k=k, filters=_parse_filters(filters))
    typer.echo(json.dumps([c.model_dump() for c in chunks], ensure_ascii=False, indent=2))

@app.command("summarize")
def summarize(document=None, query=None, filters=None, k=None, output=None, fmt="text"):
    result = summarize_learning(document=document, query=query, filters=_parse_filters(filters), k=k)
    _emit(result, output, fmt)
```

CLI hiện hỗ trợ các nhóm lệnh chính: `ingest` để index PDF trong `data/`, `ask` để hỏi đáp, `debug-retrieval` để kiểm tra retrieval không gọi LLM, và các lệnh `summarize`, `quiz`, `flashcards` để tạo học liệu và xuất kết quả dưới dạng text, JSON hoặc Markdown.

Giao diện web được cài đặt bằng Streamlit trong `src/interfaces/ui.py`. UI không gọi trực tiếp Qdrant hoặc LLM, mà gửi HTTP request đến FastAPI backend bằng `httpx`. Phần định dạng giao diện được tách sang `src/interfaces/styles.py`, giúp code UI gọn hơn.

src/interfaces/ui.py

```python
_API = settings.api_url

def _api(method, path, **kwargs):
    return httpx.request(method, f"{_API}{path}", timeout=120, **kwargs)

def run():
    st.set_page_config(page_title="RAG Learning System", layout="wide")
    st.markdown(GLOBAL_CSS, unsafe_allow_html=True)

    filenames, page = _sidebar()
    tabs = st.tabs(["Hỏi đáp", "Tóm tắt", "Quiz", "Flashcards"])

    for tab, fn in zip(tabs, [_tab_chat, _tab_summary, _tab_quiz, _tab_flashcards]):
        with tab:
            fn(filenames, page)
run()
```

Về mặt bố cục, giao diện Streamlit gồm thanh bên để tải PDF, xem danh sách tài liệu và chọn phạm vi; các tab chức năng cho hỏi đáp, tóm tắt, quiz và flashcards; cùng khu vực tải kết quả ở dạng JSON hoặc Markdown.

IV. Thực nghiệm

Trong phần này, chúng ta thực hiện quy trình thực nghiệm nhằm đánh giá hiệu quả của hệ thống thông qua bộ đánh giá Retrieval-augmented generation assessment (Ragas). Mục tiêu là tìm ra cấu hình tối ưu để đảm bảo hệ thống cung cấp thông tin chính xác và tin cậy cho người học.

IV.1. Retrieval-augmented generation assessment

Ragas bao gồm 4 metrics khác nhau, chia thành hai khía cạnh chính: quá trình truy xuất (Retrieval) và quá trình tạo văn bản (Generation):

- **Context Recall (Độ bao phủ ngữ cảnh):** Đánh giá việc hệ thống có tìm đủ thông tin hay không. Điểm số cao nghĩa là hệ thống đã trích xuất thành công và đầy đủ tất cả các dữ kiện cần thiết từ cơ sở dữ liệu để có thể giải quyết câu hỏi.
- **Context Precision (Độ chính xác ngữ cảnh):** Đánh giá việc hệ thống có tìm thừa thông tin (nhiễu) hay không. Điểm số cao nghĩa là hệ thống chỉ trích xuất đúng các đoạn văn bản hữu ích và có khả năng xếp hạng những đoạn quan trọng nhất lên vị trí đầu tiên.
- **Faithfulness (Tính trung thực):** Kiểm tra xem câu trả lời được tạo ra có hoàn toàn dựa trên ngữ cảnh đã truy xuất hay không, nhằm phát hiện hiện tượng hallucination.
- **Answer Relevance (Độ liên quan câu trả lời):** Đánh giá mức độ phù hợp của câu trả lời đối với truy vấn ban đầu của người dùng.

Thay vì phụ thuộc vào việc đánh giá và gán nhãn thủ công từ con người vốn tốn kém thời gian và dễ mang tính chủ quan, quá trình đo lường các chỉ số Ragas trên được tự động hóa thông qua phương pháp LLM-as-judge, sử dụng mô hình LLM để đánh giá.

src/evaluation/ragas_evaluator.py

```python
from ragas import evaluate
from ragas.embeddings import LangchainEmbeddingsWrapper
from ragas.llms import LangchainLLMWrapper
from ragas.metrics import answer_relevancy, context_precision, context_recall,
                                            faithfulness
from ragas.run_config import RunConfig

def get_ragas_metrics(llm, embeddings):
    faithfulness.llm = llm
    answer_relevancy.llm = llm
    context_precision.llm = llm
    context_precision.embeddings = embeddings
    context_recall.llm = llm
    context_recall.embeddings = embeddings
    return [faithfulness, answer_relevancy, context_precision, context_recall]

def run_evaluation(test_cases: list[dict[str, str]], *,
```

```python
        answer_fn: Callable[[str], RagAnswer] = default_answer_fn,
        llm_provider: str | None = None,
        timeout_s: int = 180, max_retries: int = 3, max_workers: int = 4,):

    data: dict[str, list] = {
        "user_input": [], "response": [],
        "retrieved_contexts": [], "reference": [],
    }
    for case in test_cases:
        rag_response = answer_fn(case["question"])
        data["user_input"].append(case["question"])
        data["response"].append(rag_response.answer)
        data["retrieved_contexts"].append([chunk.text for chunk in rag_response.chunks
                                           ])
        data["reference"].append(case["ground_truth"])

    eval_dataset = Dataset.from_dict(data)

    llm = LangchainLLMWrapper(get_llm(provider=llm_provider))
    embeddings = LangchainEmbeddingsWrapper(get_embeddings())
    metrics = get_ragas_metrics(llm, embeddings)
    config = RunConfig(timeout=timeout_s, max_retries=max_retries, max_workers=
                                            max_workers,
    )
    return evaluate(dataset=eval_dataset, metrics=metrics, llm=llm, embeddings=
                                            embeddings, run_config=config)
```

Lưu ý: Để quy trình đánh giá Ragas có thể vận hành và tính toán chính xác các chỉ số, hệ thống bắt buộc phải được cung cấp một tập dữ liệu kiểm thử tiêu chuẩn. Tập dữ liệu này chứa các cặp question và đáp án ground truth chuẩn xác để mô hình LLM tiến hành đối chiếu, phân tích mức độ sai lệch của hệ thống (Bảng 1).

Bảng 1: Ví dụ tập dữ liệu đánh giá mẫu (Evaluation Dataset) phục vụ đo lường Ragas

No.	Question	Ground Truth
1	Kỹ thuật Supervised Fine-Tuning (SFT) giúp mô hình học được điều gì ngoài việc biết "trả lời cái gì"?	SFT còn dạy mô hình "trả lời như thế nào", bao gồm việc trả lời đúng vai trò, đúng cấu trúc và kiểu nội dung bài toán yêu cầu.
2	Ba vai trò (role) quen thuộc trong định dạng hội thoại của dữ liệu instruction-tuned là gì?	Dữ liệu được chuyển sang định dạng hội thoại gồm ba vai trò quen thuộc là system, user và assistant.
3	Ý tưởng cốt lõi của phương pháp LoRA là gì?	LoRA giữ nguyên trọng số gốc và chỉ học thêm một phần hiệu chỉnh nhỏ có cấu trúc hạng thấp.

IV.2. Chiến lược thực nghiệm

Chiến lược thực nghiệm dựa trên tối ưu hóa recall trước. Cơ sở của chiến lược này xuất phát từ bản chất của kiến trúc RAG: sự chính xác của quá trình tạo sinh nội dung phụ thuộc hoàn toàn vào độ bao phủ của ngữ cảnh được trích xuất. Nếu chỉ số context recall thấp, hệ thống sẽ thất bại trong việc thu thập đủ dữ kiện cần thiết, điều này trực tiếp khiến mô hình ngôn ngữ lớn thiếu căn cứ suy luận và sinh ra các phản hồi sai lệch. Để cải thiện chỉ số này, các kỹ thuật thường được áp dụng bao gồm tối ưu hóa Chunking, áp dụng Query Expansion hoặc Hybrid Search.

Sau khi lượng thông tin cung cấp đã đạt độ bao phủ cần thiết, trọng tâm tối ưu mới là tinh chỉnh context precision nhằm giảm thiểu lượng thông tin dư thừa và nhiễu. Các phương pháp có thể sử dụng bao gồm sử dụng Reranker, Metadata Filtering hoặc Context Compression. Sự cải thiện về chất lượng ngữ cảnh ở bước này sẽ tạo tiền đề gián tiếp thúc đẩy hai chỉ số ở giai đoạn Generation. Trong đó Faithfulness có thể được cải thiện thông qua Prompt Engineering và Answer Relevance có thể tối ưu qua các kỹ thuật Query Rewriting.

Trong giới hạn phạm vi của dự án này, thực nghiệm chỉ tập trung đi sâu vào hai kỹ thuật: Tối ưu hoá Chunking và áp dụng mô hình Reranker ở giai đoạn truy xuất.

IV.3. Thực nghiệm: Chunking

Trong phần này, chúng ta sẽ thực nghiệm hai phương pháp chia nhỏ văn bản phổ biến trong RAG là Recursive Chunking và Semantic Chunking. Mục tiêu là tìm ra cấu hình chunking tối ưu giúp cân bằng giữa độ chính xác ngữ cảnh và khả năng bao phủ thông tin:

- **Recursive Chunking**: Chia nhỏ văn bản dựa trên độ dài cố định kết hợp với cắt tại các ký tự phân cách.
- **Semantic Chunking**: Chia văn bản dựa trên độ tương đồng về ngữ nghĩa giữa các câu, sử dụng một embedding model để gom cụm các câu có nội dung liên quan thành một chunk.

IV.3.1. Recursive Chunking

Đối với phương pháp Recursive Chunking, thực nghiệm tiến hành đánh giá trên các cấu hình kích thước chunk và độ chồng lấp chunk. Các thiết lập cụ thể bao gồm: 500/50, 800/100, 1000/150 và 1500/ 200.

src/evaluation/chunking_strategy.py

```python
from dataclasses import dataclass
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter

DEFAULT_SEPARATORS = ["\n\n", "\n", ". ", " ", ""]

```

```
_RECURSIVE_CONFIGS = [
("rc_500_50", 500, 50),
("rc_800_100", 800, 100),
("rc_1000_150", 1000, 150),
("rc_1500_200", 1500, 200),
]

@dataclass(frozen=True)
class ChunkingStrategy:
    strategy_id: str
    chunker: object
    params: dict[str, object]

@dataclass(frozen=True)
class RecursiveChunker:
    chunk_size: int = 500
    chunk_overlap: int = 50
    separators: list[str] | None = None

def _splitter(self) -> RecursiveCharacterTextSplitter:
    return RecursiveCharacterTextSplitter(
        chunk_size=self.chunk_size,
        chunk_overlap=self.chunk_overlap,
        separators=self.separators or DEFAULT_SEPARATORS,
        is_separator_regex=False,
    )

def split_documents(self, documents: list[Document]) -> list[Document]:
    if not documents:
        return []
    return self._splitter().split_documents(documents)
```

IV.3.2. Semantic Chunking

Thay vì cắt văn bản cơ học theo số lượng ký tự, Semantic Chunking tính toán vector ngữ nghĩa của từng câu. Thực nghiệm sử dụng ba tiêu chí ngưỡng để quyết định điểm tách đoạn: theo phân vị (percentile), theo độ lệch chuẩn (standard deviation) và theo khoảng tứ phân vị (interquartile).

src/evaluation/chunking_strategy.py

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_core.embeddings import Embeddings

_SEMANTIC_CONFIGS = [
("semantic_percentile", "percentile"),
("semantic_std_dev", "standard_deviation"),
("semantic_interquartile", "interquartile"),
]

@dataclass(frozen=True)
```

```python
class SemanticChunkerWrapper:
    """Wrapper for LangChain SemanticChunker."""

    embeddings: Embeddings
    breakpoint_type: str = "percentile"

    def _splitter(self) -> SemanticChunker:
        return SemanticChunker(
            embeddings=self.embeddings,
            breakpoint_threshold_type=self.breakpoint_type,
        )

    def split_documents(self, documents: list[Document]) -> list[Document]:
        if not documents:
            return []
        return self._splitter().split_documents(documents)

    def split_text(self, text: str) -> list[str]:
        return self._splitter().split_text(text)
```

Toàn bộ quy trình thực nghiệm được tự động hóa thông qua file `run_chunking.py`. Cụ thể, hàm `_evaluate_strategy` sẽ lặp qua từng cấu hình sau đó tiến hành đánh giá trên tập dữ liệu benchmark, và xuất kết quả đầu ra.

src/evaluation/run_chunking.py

```python
def _evaluate_strategy(
    strategy: ChunkingStrategy, output_dir: Path, test_cases: list[dict]
    ) -> dict[str, object]:
    collection_name = f"{settings.qdrant_collection}__{strategy.strategy_id}"
    chunk_count = ingest(recreate=True, collection_name=collection_name, chunker=
                                            strategy.chunker)

    result_out: dict[str, object] = {
        "strategy_id": strategy.strategy_id,
        "chunk_count": chunk_count,
        "summary_metrics": {},
    }

    try:
        def answer_fn(q: str) -> RagAnswer:
            return answer(q, collection_name=collection_name)

        result = run_evaluation(test_cases, answer_fn=answer_fn, llm_provider="vllm")
        df = result.to_pandas()
        result_out["summary_metrics"] = summary_metrics(df)

    except Exception as exc:
        result_out["error"] = str(exc)

    write_json(output_dir / f"{strategy.strategy_id}.json", result_out)
    return result_out
```

IV.3.3. Kết quả đánh giá

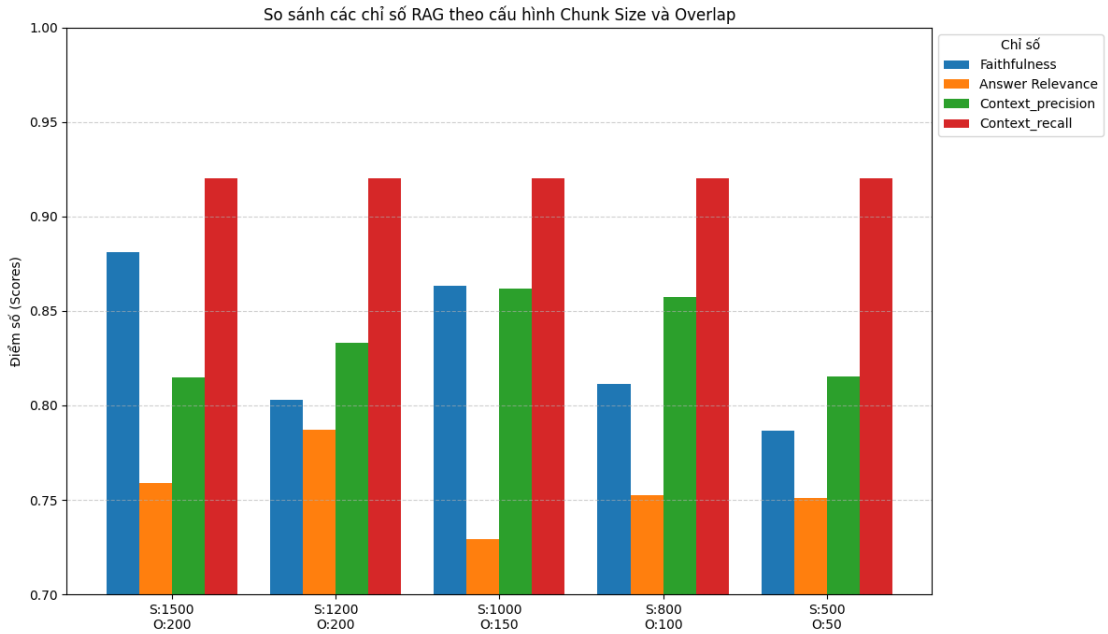
Kết quả chi tiết từ thực nghiệm được tổng hợp trong bảng sau:

Bảng 2: Kết quả đánh giá các chiến lược Chunking trên bộ tập dữ liệu Benchmark

Method	Config	Chunks	Faith.	Relev.	Prec.	Recall
Recursive	1500/200	246	0.8811	0.7590	0.8149	0.92
	1000/150	340	0.8632	0.7292	**0.8618**	**0.92**
	800/100	401	0.8112	0.7526	0.8572	0.92
	500/50	600	0.7867	0.7509	0.8151	0.92
Semantic	Interquartile	564	**0.8867**	**0.7959**	0.8117	0.92
	Std_dev	291	0.8693	0.7578	0.8040	0.90
	Percentile	465	0.8573	0.7344	0.8225	0.90

Phân tích kết quả thực nghiệm:

- **Recursive Chunker:** Duy trì Context Recall ổn định ở mức 0,92 trên mọi thiết lập. Trong đó, cấu hình 1000/150 đạt Context Precision tối ưu (0,8618) , trong khi thiết lập 500/50 gây phân mảnh ngữ cảnh, khiến Faithfulness giảm xuống 0,7867.
- **Semantic Chunker:** Phương pháp Interquartile ghi nhận Faithfulness cao nhất thực nghiệm (0,8867), nhưng các biến thể Std_dev và Percentile lại có Recall thấp hơn (0,90) so với nhóm Recursive.

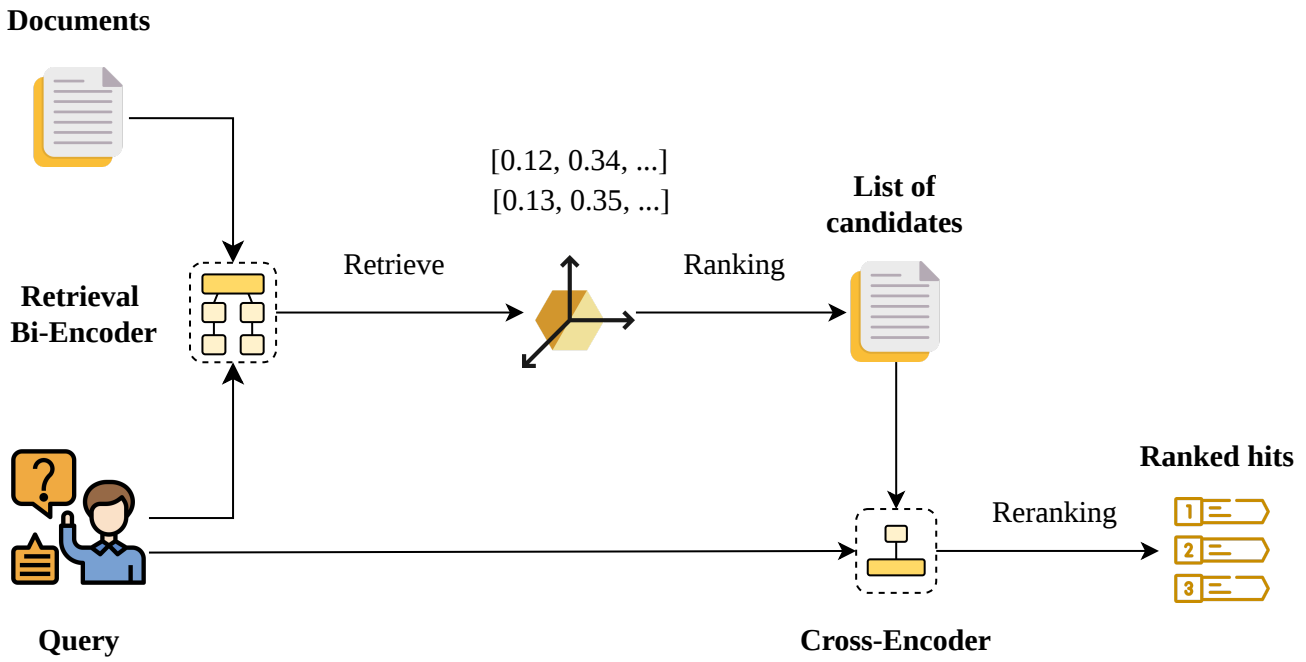


Hình 7: Đồ thị so sánh các chỉ số Ragas theo các cấu hình Recursive Chunking khác nhau.

Kết luận: Qua phân tích, cấu hình Semantic Interquartile có chỉ số Faithfulness và Answer Relevance cao nhất đang là thiết lập tốt nhất để tối ưu hóa trực tiếp chất lượng câu trả lời. Tuy nhiên, mục tiêu của giai đoạn thực nghiệm hiện tại là tìm ra một mô hình truy xuất có độ bao phủ và độ chính xác ngữ cảnh cao làm nền tảng. Do đó, cấu hình **Recursive Chunker (1000/150)** được lựa chọn làm thiết lập tiêu chuẩn vì nó giữ được Context Precision cao nhất (0.8618) và Context Recall đạt 0.92. Những điểm yếu về Answer Relevance và Faithfulness được kỳ vọng khắc phục khi chúng ta áp dụng thêm bước Reranking bằng Cross-Encoder ở thực nghiệm tiếp theo.

IV.4. Thực nghiệm: Reranking

Sau khi xác định được cấu hình chia đoạn văn bản tối ưu là Recursive Chunker (1000/150), thực nghiệm tiếp theo tập trung vào việc tối ưu hóa Context Precision thông qua kỹ thuật Reranking.



Hình 8: Sơ đồ Reranking sử dụng Cross-Encoder.

Kiến trúc RAG truyền thống thường gặp hạn chế khi chỉ dựa vào độ tương đồng vector, dễ dẫn đến việc thu thập các đoạn văn bản có từ vựng giống nhau nhưng thiếu liên quan về ngữ nghĩa thật sự. Để giải quyết vấn đề này, một mô hình Cross-Encoder đã được tích hợp vào sau quá trình trích xuất ngữ cảnh liên quan, như Hình 8:

- **Cơ chế Reranking với Cross-Encoder:** Mô hình encoder `BAAI/bge-reranker-v2-m3` được sử dụng để kết hợp trực tiếp câu truy vấn và từng đoạn văn bản ngữ cảnh thành một chuỗi thống nhất. Mạng Transformer sau đó sẽ thực hiện cơ chế cross-attention để đánh giá độ liên quan.

- **Chiến lược truy xuất:** Do chi phí tính toán của Cross-Encoder lớn, ta chia việc truy vấn hai giai đoạn. Đầu tiên, không gian tìm kiếm được giới hạn bằng cách truy xuất 15 đoạn ngữ cảnh tiềm năng nhất (`initial_k = 15`). Cross-Encoder sau đó sẽ tính toán lại điểm tương quan giữa 15 đoạn này với câu truy vấn. Dựa trên điểm số mới, hệ thống chắt lọc ra 5 đoạn có độ chính xác ngữ nghĩa cao nhất (`rerank_k = 5`) làm ngữ cảnh cuối cùng, giúp mô hình ngôn ngữ lớn tạo sinh câu trả lời bám sát thực tế và hạn chế hallucination.

IV.4.1. Cài đặt Reranker

Chúng ta triển khai cơ chế Reranking trong pipeline RAG thông qua hàm `answer_with_reranker`

src/evaluation/run_reranking.py

```python
from sentence_transformers import CrossEncoder
from src.rag import ANSWER_TEMPLATE, format_citations, render_prompt, retrieve
from src.schemas import RagAnswer
from src.llm import invoke_llm

RERANKER_MODEL = "BAAI/bge-reranker-v2-m3"

def answer_with_reranker(
    question: str,
    collection_name: str,
    reranker: CrossEncoder,
    initial_k: int = 15,
    rerank_k: int = 5,
    filters: dict[str, object] | None = None,
) -> RagAnswer:
    # Giai đoạn 1: Truy xuất thô (initial_k)
    chunks = retrieve(question, k=initial_k, filters=filters, collection_name=
                                            collection_name)
    if not chunks:
        return RagAnswer(
            question=question,
            answer="Tôi không có đủ thông tin trong ngữ cảnh được cung cấp để trả lời."
                                                ,
        )

    # Giai đoạn 2: Tính toán điểm số tương quan chéo bằng Cross-Encoder
    scores = reranker.predict([[question, chunk.text] for chunk in chunks])
    for chunk, score in zip(chunks, scores):
        chunk.score = float(score)

    # Xếp hạng lại và lọc ra các đoạn liên quan nhất (rerank_k)
    reranked = sorted(chunks, key=lambda c: c.score, reverse=True)[:rerank_k]

    # Đưa ngữ cảnh đã được lọc vào prompt cho LLM
    prompt = render_prompt(ANSWER_TEMPLATE, question=question, chunks=reranked)
    text = invoke_llm(prompt)

    return RagAnswer(
```

```
        question=question,
        answer=text.strip(),
        citations=format_citations(reranked),
        chunks=reranked,
    )
```

Các bước đánh giá bằng Ragas sau đó được thực hiện như phần đánh giá Chunking.

IV.4.2. Kết quả đánh giá

Kết quả thu được từ bộ chỉ số Ragas khi áp dụng Reranker cho thấy sự cải thiện vượt bậc và toàn diện trên cả 4 khía cạnh đo lường so với mô hình baseline (Bảng 3).

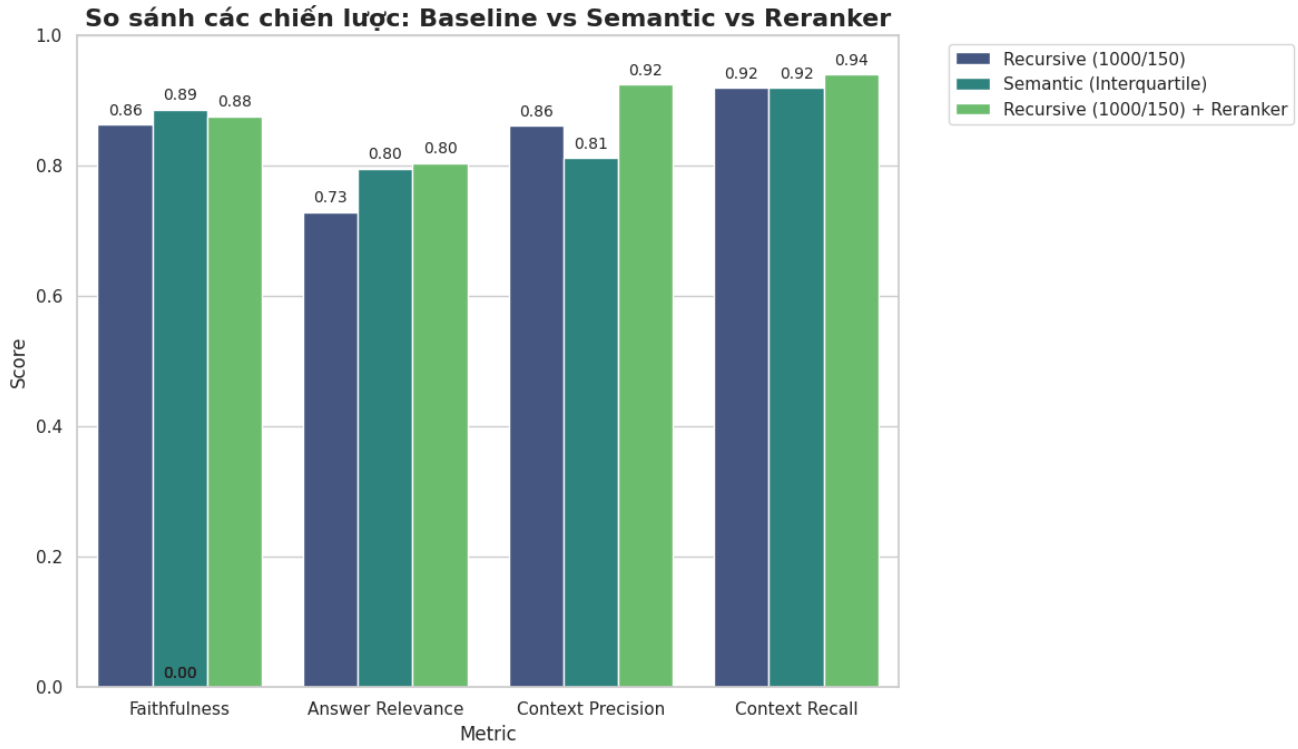
Bảng 3: So sánh hiệu suất của cấu hình tốt nhất khi có và không có Reranker.

Phương pháp	Chunks	Faithfulness	Answer Relevance	Context Precision	Context Recall
Recursive (1000/150)	340	0.8632	0.7292	0.8618	0.9200
Semantic (Interquartile)	564	0.8867	0.7959	0.8117	0.9200
Recursive + Reranker	**340**	**0.8761**	**0.8044**	**0.9247**	**0.9400**

Phân tích kết quả thực nghiệm

Context Precision tăng lên 0.92 chứng minh mô hình Reranker đã loại bỏ thành công các ngữ cảnh nhiễu so với ban đầu. Đồng thời, Context Recall cũng đạt mức 0.94. Sự gia tăng Recall này không xuất phát từ bản chất của mô hình Reranker mà là kết quả của chiến lược mở rộng không gian tìm kiếm, đảm bảo hệ thống gần như không bỏ sót bất kỳ dữ kiện quan trọng nào.

Nhờ việc cung cấp cho LLM một tập ngữ cảnh đầu vào vô cùng sạch và cô đọng, mô hình ngôn ngữ không còn bị phân tâm. Điểm Answer Relevance tăng mạnh từ 0.7292 lên 0.8044, đồng thời Faithfulness cũng đạt mức 0.8761. Điều này giúp giảm thiểu tối đa rủi ro hallucination, đảm bảo mô hình trả lời đúng trọng tâm.



Hình 9: Đồ thị so sánh các phương pháp thực nghiệm Chunking và Reranking.

Kết luận: Sự kết hợp giữa **Recursive Chunker (1000/150)** và bước **Reranking** bằng Cross-Encoder là cấu hình mạnh mẽ nhất, không chỉ tối ưu hóa khả năng truy vấn ngữ cảnh mà còn đảm bảo chất lượng phản hồi cuối cùng của LLM bám sát chặt chẽ với nhu cầu của người dùng.

Phụ lục

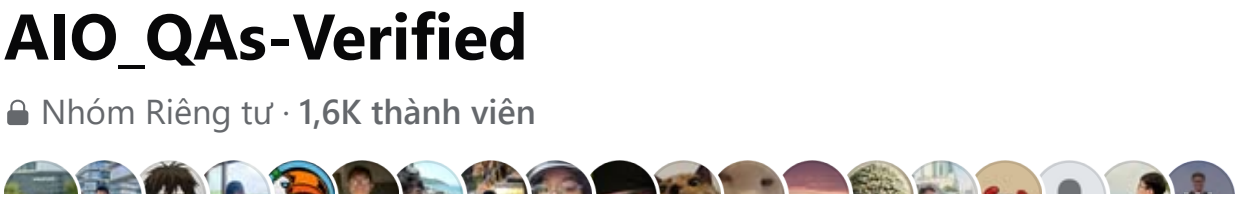
1. **Resources:** Các file dataset, code gợi ý (hint) và bài giải (solution) có thể được tải tại đây. (Lưu ý: Sau khi hết deadline của phần bài này, ad mới copy các nội dung bài giải vào đường dẫn).

2. **Demo:** Chương trình demo cho nội dung trong bài có thể được truy cập tại đây.

3. **Rubric:**

Mục	Kiến Thức	Đánh Giá
II.	- Kiến thức tổng quan về hệ thống học tập dựa trên RAG. - Quy trình xây dựng kho tri thức từ tài liệu: Tiền xử lý, chunking, embedding. - Nguyên lý hoạt động của pipeline truy xuất và sinh phản hồi.	- Nắm vững khái niệm về kiến trúc RAG và bài toán hỏi đáp trên tài liệu cá nhân. - Hiểu rõ luồng hoạt động tổng thể từ khi nạp dữ liệu đến khi trả kết quả cho người dùng.
III.	- Các kỹ thuật chunking và gắn metadata. - Cách cài đặt và sử dụng vector database (Qdrant) để lưu trữ và truy xuất ngữ nghĩa. - Kỹ thuật tích hợp các mô hình ngôn ngữ lớn (LLM) qua API hoặc máy chủ cục bộ (Local). - Thuật toán xây dựng các chức năng học tập: Hỏi đáp, Tóm tắt (Map-Reduce), Trắc nghiệm và Thẻ ghi nhớ.	- Cài đặt thành công mã nguồn tiền xử lý và lưu trữ dữ liệu vào Qdrant. - Xây dựng được pipeline kết nối cơ sở dữ liệu vector với LLM. - Lập trình và kiểm thử thành công các tính năng sinh học liệu tự động.
IV.	- Phương pháp đánh giá hệ thống RAG sử dụng bộ chỉ số Ragas (Context Recall, Context Precision, Faithfulness, Answer Relevance). - Cách thiết lập thực nghiệm để so sánh các chiến lược Chunking (Recursive vs. Semantic). - Kỹ thuật Reranking sử dụng mô hình Cross-Encoder để tối ưu hóa độ chính xác ngữ cảnh.	- Biết cách vận hành quy trình đánh giá tự động bằng phương pháp LLM-as-judge. - Đo lường và phân tích được tác động của các tham số Chunking lên hiệu suất hệ thống. - Cài đặt thành công mô hình Reranker để cải thiện chất lượng truy xuất và giảm thiểu hallucination cho LLM.

4. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.



Hình 10: Hình ảnh group facebook AIO Q&A.